A Reusable Explainable Machine Learning Framework for Cyber Attack Detection

Krishna Sharma K
Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
krishnasharma.k2023@vitstudent.ac.in

Abhinav Renjith
Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
abhinav.renjith2023@vitstudent.ac.in

Tamilarasi Kathirvel Murugan
Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
tamilarasi.k@vit.ac.in

***Abstract*—The use of machine learning in network intrusion detection systems is often challenging because there isn't transparency in how these systems work and because the ways to test them are not consistent. This research presents a framework that is easy to reuse, made up of separate parts and can explain its decisions. The framework focuses on making sure the predictions are strong and that the decisions are clear. It uses the CIC-IDS2017 dataset. The system breaks the machine learning process into five parts that can be swapped out: data cleaning choosing features based on mutual information building models with tree-based methods checking how well the models work with unbalanced data and using SHAP to explain the results. It tests three models: Random Forest, XGBoost and LightGBM. The tests include ways to check how good the models are, like splitting the data into five parts using a method to see how stable the results are making sure the probabilities are correct and checking how well the models handle changes. The tests show that Random Forest has the best performance with an F1-Score of 0.9976 and an MCC of 0.9963. Xgboost is the best for real time use on edge devices. It can handle 825,582 samples each second and only needs 231.3 KB of space. When looking at how consistent the explanations are, XGBoost had the highest score, with a Spearman ρ of 0.9821. When tested against noise, Random Forest was the most resistant.**

***Index Terms*—Explainable machine learning, intrusion detection, CIC-IDS2017, SHAP, ensemble learning, feature selection, cybersecurity, class imbalance**

I. Introduction

Nowadays, computer networks experience frequent contacts with complex cyber threats. This makes effective Intrusion Detection Systems necessary for protecting digital infrastructure. Traditional signature based systems frequently fail when they are exposed to new or changing attacks. Machine learning techniques are newer, more effective methods that identify network traffic that deviates from the norm. To achieve this, predictive models analyze data from complex network flows, making it possible for these models to identify the differences between normal activity and unauthorized entries. Predictive models are capable of performing the identifications with a high degree of mathematical precision.

Despite these advancements, the practical adoption of machine learning in security operations is frequently hindered by two major challenges: a lack of model transparency and a reliance on brittle, single-use experimental setups. Many existing intrusion detection studies focus almost exclusively on maximizing overall accuracy, a metric that is notoriously misleading given the severe class imbalance inherent in real-world network traffic. Moreover, these models are often deployed as opaque black boxes. This leaves security analysts without the necessary reasoning to understand why a specific network flow was flagged as an attack. Even highly accurate models remain difficult to trust and operationalize without clear explainability and reproducible, standardized pipelines.

To address the research gap in this area, this study proposes a modular predictive analytics framework that can be reused and modified to detect cyber attacks. The system employs the CIC-IDS2017 dataset to provide clear reasoning behind its outputs, beyond functioning as a simple classifier that calculates a single numerical result. The project shows how every stage of the predictive analytics pipeline works together. The framework creates a standard for the crucial steps of the machine learning lifecycle, including data preprocessing and reusable feature selection based on mutual-information. It also includes procedures to train Random Forest, XGBoost and LightGBM models, ensemble models that use decision trees to identify patterns in network traffic, parallelly.

The system measures performance using the Matthews Correlation Coefficient (MCC) and F1-Score, metrics that are appropriate for datasets where categories are imbalanced. The use of those specific metrics ensures that the assessment of system threat detection is accurate, and it avoids the error of majority-class bias. To make the algorithm's decision-making process transparent, the framework includes SHapley Additive exPlanations (SHAP). SHAP provides information about which feature importance across the entire model. It also provides specific reasons for every alert that the system generates. The study focuses on building a architecture that is easy to modify and clear to understand, rather than creating a completely novel algorithm from scratch. This study provides a practical foundation that helps researchers and security analysts compare different models in a consistent and reproducible way, while developing confidence in tools that automate the protection of computer systems.

The CIC-IDS2017 benchmark was developed to characterize contemporary intrusion traffic, while practical intrusion detection must also account for the limits of machine learning under real-world conditions [21], [22]. Surveys of data-mining

methods for cybersecurity and network intrusion-detection datasets emphasize the importance of selecting suitable benchmarks and applying reproducible evaluation protocols [23], [24]. For transparent prediction, SHAP provides a unified approach to interpreting model outputs [25], while XGBoost and LightGBM provide scalable gradient-boosting systems for comparative evaluation of predictive quality and deployment efficiency [26], [27].

II. Related Work and Literature Survey

1. Gaspar, D., Silva, P., & Silva, C. (2024). Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron. IEEE Access, 12, 30164–30175.

Machine learning IDS models have grown in predictive power but remain opaque, which limits analyst trust and slows adoption in security operations. The authors frame the problem as the mismatch between a black-box Multi-Layer Perceptron's classification performance and a security analyst's need to justify each alert. Their methodology trains an MLP-based IDS on IoT-oriented network traffic and then applies two widely used post-hoc explanation methods, LIME and SHAP, comparing the local and global explanations each produces for the same predictions. They further run a perturbation analysis to test how stable the explanations remain when input features are slightly altered. The results show that SHAP delivers more consistent global feature-attribution rankings across repeated runs, while LIME is faster per-instance but less stable under perturbation. The paper concludes that combining both explainers, rather than relying on one, gives security teams a more complete and trustworthy picture of why the MLP flags a given flow as malicious. It supports the project's plan of pairing SHAP with feature-importance outputs rather than using a single explanation technique alone.

2. Arreche, O., Guntur, T. R., Roberts, J. W., & Abdallah, M. (2024). E-XAI: Evaluating Black-Box Explainable AI Frameworks for Network Intrusion Detection. IEEE Access, 12, 23954–23988.

The paper opens from the observation that dozens of XAI techniques now exist for intrusion detection, but there is no standard way to judge which explanation method is actually the most useful or faithful for a given model and dataset. The core problem is therefore evaluation itself: how should a practitioner choose among competing explainers such as SHAP, LIME, and others when building an IDS. The proposed E-XAI framework benchmarks multiple XAI methods against several black-box classifiers trained on standard intrusion detection benchmarks, scoring the explanations on criteria such as fidelity, consistency, and computational cost rather than judging classifier accuracy alone. Their experiments span multiple datasets and model families to keep the comparison broad. The results indicate that no single explainer dominates across every metric: SHAP-based methods tend to score highest on faithfulness to the underlying model, while simpler surrogate-based methods are cheaper to compute but less reliable on complex ensembles. The authors argue that framework-level, criteria-based evaluation should become standard practice before an XAI method is adopted operationally. This directly informs how the coursework project should justify choosing SHAP as its primary explainer rather than adopting it by default.

3. Ozkan-Okay, M., Akin, E., Aslan, O., Kosunalp, S., Iliev, T., Stoyanov, I., & Beloev, I. (2024). A Comprehensive Survey: Evaluating the Efficiency of Artificial Intelligence and Machine Learning Techniques on Cyber Security Solutions. IEEE Access, 12, 12229–12256.

This survey addresses the broad and fast-moving landscape of AI and ML techniques applied to cybersecurity, noting that practitioners often lack a consolidated view of which algorithm families suit which threat categories. The problem statement centers on fragmentation: many isolated studies report high accuracy on narrow datasets without comparable methodology or evaluation criteria across attack types such as malware, phishing, intrusion, and spam. The authors' approach is a structured literature synthesis that categorizes techniques (traditional ML, deep learning, ensemble methods) by cybersecurity application domain, and cross-tabulates reported strengths, weaknesses, and typical evaluation metrics such as accuracy, precision, recall, and F1-score. Rather than new experiments, the contribution is a comparative efficiency assessment drawn from published numerical results across the surveyed works. The synthesis finds that ensemble and hybrid ML approaches (e.g., Random Forest and boosting variants) consistently outperform single classical classifiers on intrusion-detection-style tasks, while deep learning gains are more domain-dependent and costlier to deploy. The survey's practical takeaway — that ensemble tree-based methods offer the best accuracy-to-complexity trade-off for network intrusion detection — supports the project's choice of Random Forest, XGBoost, and LightGBM as the core comparison set.

4. Wang, Z., Li, F., & Zhang, Y. (2024). Enhancing Intrusion Detection in VANETs Through Stacked Ensemble Learning. IEEE Access, 12, 10123–10135.

Vehicular ad-hoc networks (VANETs) face intrusion risks that differ from conventional wired networks because of high mobility and intermittent connectivity, and the authors note that single classifiers often struggle to generalize across such variable conditions. The problem addressed is that individual ML models trained on VANET traffic show inconsistent detection performance across different attack types, motivating the need for a model that can combine multiple learners' strengths. Their methodology constructs a stacked ensemble that combines several base classifiers with a meta-learner trained on the base models' outputs, following a standard preprocessing and feature-encoding pipeline typical of network intrusion detection studies. The stacked model is benchmarked against its individual base learners on VANET intrusion data using accuracy, precision, recall, and F1-score. Results show the stacked ensemble improves detection accuracy and reduces false positives compared with any single base classifier, particularly for attack classes that individual models handled inconsistently. The paper illustrates, in a different network

context, the same principle the coursework project intends to apply generically: combining multiple trained models within a reusable pipeline yields more robust detection than any single algorithm.

5. Birahim, S. A., Paul, A., Rahman, F., Islam, Y., Roy, T., Hasan, M. A., Haque, F., & Chowdhury, M. E. (2025). Intrusion Detection for Wireless Sensor Network Using Particle Swarm Optimization Based Explainable Ensemble Machine Learning Approach. IEEE Access.

Wireless Sensor Networks (WSNs) are resource-constrained and prone to security breaches, and the authors identify a gap in prior WSN-IDS research: most models either optimize purely for accuracy or provide explanations, rarely both together. The problem statement is thus twofold — achieving high detection accuracy for WSN intrusions while also making the ensemble's decisions interpretable to network operators with limited computational overhead. Their proposed method uses Particle Swarm Optimization (PSO) to tune the hyperparameters of an ensemble of machine learning classifiers, and layers SHAP-based explainability on top of the optimized ensemble to expose which sensor-traffic features drive each classification. The pipeline includes standard preprocessing of WSN traffic features before optimization and training. Reported results show the PSO-tuned explainable ensemble achieving detection accuracy in the high 90s percent range while outperforming non-optimized baseline ensembles, with SHAP plots confirming that features tied to packet timing and node behavior dominate the model's decisions. The paper is a close analogue of the project's own goal — pairing hyperparameter-tuned ensembles with SHAP explanations — but for a resource-constrained network setting rather than CIC-IDS2017.

6. Janabi, A., et al. (2024). Survey: Intrusion Detection System in Software-Defined Networking. IEEE Access, 12, 164097–164120.

Software-Defined Networking (SDN) centralizes control logic, which changes both the attack surface and the opportunities for intrusion detection compared with traditional networks. The paper's problem framing is that SDN-specific IDS research has grown quickly but is scattered across many architectures (centralized, distributed, hybrid controllers) and ML techniques, making it hard to identify best practices. As a survey, the methodology is a systematic review and taxonomy of SDN-IDS approaches, categorizing them by detection technique (signature-based, anomaly-based, ML/DL-based), dataset used, and deployment point within the SDN control/data plane. The authors compile reported performance figures from the surveyed studies, covering accuracy, detection latency, and scalability under SDN traffic loads, including work that uses CIC-IDS2017-style benchmarks adapted to SDN testbeds. Their synthesis finds that ML-based anomaly detection integrated at the SDN controller level generally achieves the best trade-off between detection accuracy and response latency, though most surveyed systems still lack standardized reproducible evaluation protocols. This reinforces the project's decision to keep preprocessing and evaluation explicit and dataset-agnostic, since the survey highlights reproducibility as a persistent weakness in the wider IDS literature.

7. Altalhan, M., Algarni, A., & Alouane, M. T.-H. (2025). Imbalanced Data Problem in Machine Learning: A Review. IEEE Access, 13, 13686–13699.

Class imbalance is identified as one of the most persistent obstacles to reliable ML classifiers, particularly in domains like intrusion detection where malicious traffic is a small minority of total records. The problem statement is that standard classifiers trained on imbalanced data tend to be biased toward the majority (benign) class, producing misleadingly high accuracy while missing minority attack classes. The review's methodology organizes imbalance-handling techniques into data-level methods (oversampling such as SMOTE, undersampling, hybrid resampling), algorithm-level methods (cost-sensitive learning, ensemble-based approaches), and hybrid strategies, summarizing their reported effectiveness across applications including cybersecurity. The authors compile comparative results from prior studies showing that hybrid resampling combined with ensemble classifiers consistently improves minority-class recall and F1-score compared with using either resampling or a robust classifier alone, though at some cost to training time and risk of overfitting from synthetic samples. The review recommends matching imbalance-handling strategy to the severity of the imbalance ratio rather than applying oversampling uniformly. This directly supports the project's preprocessing stage, where CIC-IDS2017's heavy class imbalance across attack categories needs a deliberate, justified resampling or weighting strategy rather than a default choice.

8. Racherla, S., Sripathi, P., Faruqui, N., Kabir, M. A., Whaiduzzaman, M., & Shah, S. A. (2024). Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning. IEEE Access, 12, 63584–63597.

IoT devices generate high-volume, resource-constrained traffic streams where real-time detection latency matters as much as accuracy, and the authors note that many deep-learning IDS proposals ignore inference-time constraints. The problem addressed is building an IDS that both classifies IoT intrusions accurately and runs fast enough for near real-time deployment on IoT-scale traffic. Their proposed Deep-IDS architecture combines deep neural network layers with a lightweight preprocessing pipeline tuned for streaming IoT flow features, and evaluates the model on IoT-relevant intrusion datasets under both accuracy and latency metrics. The methodology includes systematic comparison against baseline deep learning and classical ML models under identical preprocessing to isolate the architecture's contribution. Reported results show Deep-IDS achieving high detection accuracy (above 99% in several attack categories) while keeping per-flow inference latency low enough for real-time operation, outperforming heavier baseline deep models on the latency dimension. The paper is a useful reminder for the coursework project that a reusable framework should log training and inference time alongside standard metrics, since deployability is part of what makes a detection framework genuinely usable.

9. Ahakonye, L. A. C., Nwakanma, C. I., Lee, J. M., et al. (2024). Machine Learning Explainability for Intrusion Detection in the Industrial Internet of Things. IEEE Internet of Things Magazine, 7(3), 68–74.

Industrial IoT (IIoT) systems control physical processes, so an intrusion detection error carries safety consequences beyond typical IT networks, and the authors frame explainability as essential rather than optional in this setting. The problem statement is that operators overseeing critical industrial infrastructure cannot act confidently on ML-based alerts unless they understand which measured process or network variables triggered the alert. The article's approach reviews and demonstrates the application of explainability techniques, particularly SHAP-style feature attribution, layered on top of ML classifiers trained for IIoT intrusion detection, discussing how attribution outputs should be presented to non-ML industrial operators. Rather than a single large experiment, the piece walks through representative case examples showing how feature attributions align with known IIoT attack signatures such as anomalous command injection patterns. The discussion concludes that explainability substantially increases operator trust and incident response speed compared with opaque alerting, even when it introduces a modest computational overhead. As a magazine-style IEEE IoT venue article, it complements the project's core philosophy by arguing, at a conceptual level, exactly why the "explanation" stage of the pipeline is not a cosmetic add-on but a functional requirement for real-world adoption.

10. Bhavsar, M. H., Bekele, Y. B., Roy, K., Kelly, J. C., & Limbrick, D. (2024). FL-IDS: Federated Learning-Based Intrusion Detection System Using Edge Devices for Transportation IoT. IEEE Access, 12, 52215–52226.

Transportation IoT systems distribute data across many edge devices, and centralizing all traffic for model training raises both bandwidth and privacy concerns, which the authors identify as the core motivation for a federated approach. The problem is therefore how to train an effective intrusion detector across distributed edge nodes without moving raw traffic data to a central server. Their proposed FL-IDS framework trains local models on edge devices and aggregates model updates using federated learning, comparing this decentralized setup against a conventional centralized training baseline on the same transportation IoT intrusion dataset. The methodology includes standard preprocessing for network flow features prior to federated training rounds and evaluates convergence behavior across communication rounds. Results show the federated model reaches detection accuracy close to the centralized baseline (within a few percentage points) while avoiding raw data transfer, and the paper reports the number of communication rounds needed for the federated model to stabilize. This paper broadens the reusable-framework idea from a single-machine pipeline to a distributed setting, and is a useful forward-looking reference if the coursework project's discussion section wants to note federated learning as a natural extension.

11. Altalbe, A. (2024). Enhanced Intrusion Detection in In-Vehicle Networks Using Advanced Feature Fusion and Stacking-Enriched Learning. IEEE Access, 12, 2045–2056.

In-vehicle networks (such as CAN bus systems) have very different traffic characteristics from IP networks, and the author notes that feature representations designed for conventional network intrusion detection often transfer poorly to automotive contexts. The problem addressed is improving detection accuracy for in-vehicle intrusions by better feature representation rather than by a more complex classifier alone. The proposed methodology fuses multiple feature extraction strategies into an enriched feature set and feeds this into a stacking ensemble of classifiers, following a preprocessing pipeline that normalizes and aligns heterogeneous CAN-bus-derived features before fusion. The stacked model's performance is compared against single-feature-set baselines and non-stacked classifiers on the same in-vehicle intrusion dataset. Results show that the fused-feature stacking approach improves detection accuracy and F1-score over baselines that use either stacking or feature fusion alone, indicating the two techniques are complementary rather than redundant. For the coursework project, the paper is a good illustration of why the reusable framework should treat feature engineering as a distinct, swappable pipeline stage rather than baking it into a specific model.

12. Gutti, C., Thumula, K., & Balbudhe, P. (2025). Federated Learning for Distributed IoT Security: A Privacy-Preserving Approach to Intrusion Detection. IEEE Access, 13, 135863–135875.

The paper starts from the same tension as other federated IDS work: IoT security benefits from pooling data across many devices or organizations, but doing so directly conflicts with privacy and data-ownership constraints. The problem statement focuses specifically on designing an intrusion detection approach that preserves data privacy across distributed IoT deployments while still achieving competitive detection performance compared with centralized training. Their methodology implements a federated learning pipeline where local IoT nodes or gateways train models on local traffic and share only model parameters for aggregation, with standard preprocessing applied locally before training to keep feature representations consistent across nodes. The evaluation compares the federated model's detection metrics against a centralized-training upper bound and against baseline distributed approaches without federation-specific optimizations. Results indicate the privacy-preserving federated approach achieves detection accuracy close to the centralized baseline while substantially reducing raw data movement, with the authors reporting the trade-off in accuracy as small relative to the privacy gained. This paper reinforces, from a different distributed-systems angle than paper 10 above, that the reusable ML framework the project builds could be positioned to support distributed or federated deployment as future work.

13. Albasheer, F. O., Haibatti, R. R., Agarwal, M., & Nam, S. Y. (2024). A Novel IDS Based on Jaya Optimizer and SMOTE-ENN for Cyberattacks Detection. IEEE Access, 12, 101506–101527.

The authors identify two compounding problems in network intrusion detection: high feature dimensionality that slows training and can dilute classifier performance, and class imbalance that biases classifiers toward majority (benign) traffic. Their proposed solution combines the Jaya optimization algorithm for feature selection with SMOTE-ENN (a hybrid oversampling and cleaning technique) for class balancing, applied as sequential stages before classifier training. The pipeline is tested on the UNSW-NB15 and NSL-KDD intrusion detection datasets, with the Jaya optimizer selecting the most informative feature subset and SMOTE-ENN balancing the resulting reduced feature space prior to classification. Numerical results show the combined Jaya plus SMOTE-ENN pipeline improving classification accuracy and minority-class recall over baselines that use only feature selection or only resampling, while also reducing training time due to the smaller selected feature set. The paper's two-stage design — optimization-based feature selection followed by imbalance correction — is directly transferable to the coursework project's preprocessing and feature-selection stages for CIC-IDS2017, even though the original study uses different benchmark datasets.

14. Sannigrahi, M., & Thandeeswaran, R. (2024). Predictive Analysis of Network Based Attacks by Hybrid Machine Learning Algorithms Utilizing Bayesian Optimization, Logistic Regression and Random Forest Algorithm. IEEE Access, 12, 142721.

The paper's stated goal closely mirrors the coursework project's own philosophy: moving beyond a simple train-and-report workflow toward a more rigorous predictive analytics approach for network attack detection. The problem addressed is that binary and multiclass intrusion detection tasks each benefit from different modeling strategies, and a single fixed algorithm rarely performs best across both settings. Their methodology proposes a Bayesian-Optimization-tuned Random Forest (BO_RF) for binary classification and a hybrid Logistic Regression–Random Forest (LR_RF) model for multiclass classification, evaluating both using accuracy, precision, recall, and F1-score across multiple metrics (eight for binary, twenty-six for multiclass classification across the datasets used). The experiments were implemented and evaluated in a standard notebook-based ML environment, keeping the pipeline reproducible. The results demonstrate that task-specific hybrid tuning outperforms untuned baseline classifiers in both binary and multiclass settings, supporting the authors' claim that adaptable, task-aware model selection improves robustness. This paper is one of the closest matches to the project's own approach and supports the plan to evaluate models separately for binary and multiclass CIC-IDS2017 detection tasks.

15. Khan, F. A., et al. (2024). Balanced Multi-Class Network Intrusion Detection Using Machine Learning. IEEE Access, 12, 178222.

The authors note that commonly used IDS benchmark datasets such as KDD and UNSW-NB15 are outdated and do not reflect current attack behavior, motivating their use of more recent traffic captures alongside a focus on realistic NIDS deployment architecture. The core problem is achieving reliable multi-class attack classification when the underlying dataset is imbalanced across attack categories, which is a familiar challenge for CIC-IDS2017-style data as well. Their methodology positions the IDS within a realistic network architecture (including an eBPF-based flow collection deployment) and applies data balancing techniques before training multiple ML classifiers on the balanced multi-class dataset, following a training/validation/test split (70/30 train-test, with the training portion further split 70/30 for validation). The evaluation reports classification performance per attack class rather than only in aggregate, explicitly checking whether balancing reduces majority-class bias. Results show that balancing the training data materially improves per-class recall for minority attack types without substantially reducing overall accuracy, validating the balancing step as necessary rather than optional. This paper is directly relevant to the project's own multiclass CIC-IDS2017 experiments and its emphasis on per-class reporting rather than a single aggregate accuracy number.

16. Pavithra, S., & Venkata Vikas, K. (2024). Detecting Unbalanced Network Traffic Intrusions With Deep Learning. IEEE Access, 12, 74096–74107.

Class imbalance in network traffic is again the central problem, but this paper approaches it through a hybrid architecture spanning both classical and deep models rather than resampling alone. The authors argue that no single classifier family handles all attack types equally well under imbalance, motivating a hybrid detection pipeline. Their proposed system combines XGBoost, LSTM, Mini-VGGNet, and AlexNet within a single hybrid IDS designed specifically to handle unbalanced network traffic, and additionally uses a Random Forest Regressor to rank feature importance for both improved accuracy and interpretability. The pipeline preprocesses raw traffic into structured features before routing them through the hybrid model ensemble, with the feature-importance step feeding back into the modeling stage. Reported results show the hybrid model outperforming individual baseline classifiers on unbalanced traffic, with the Random Forest Regressor's feature ranking helping to explain which traffic attributes contribute most to correct classification. This paper reinforces the value of combining a feature-importance step directly with model training, which is consistent with the project's plan to run feature selection before Random Forest, XGBoost, and LightGBM training.

17. Elsayed, S., Mohamed, K., & Madkour, M. A. (2024). A Comparative Study of Using Deep Learning Algorithms in Network Intrusion Detection. IEEE Access, 12, 58851–58870.

The paper's background motivation is the proliferation of deep learning architectures proposed for NIDS without a consistent, side-by-side comparison under identical conditions, making it hard to know which architecture is actually preferable for a given classification task. The problem addressed is thus a controlled comparative evaluation of six deep learning algorithms — DNN, CNN, RNN, LSTM, GRU, and a

hybrid CNN-LSTM — across both binary and multiclass intrusion detection using the NSL-KDD benchmark dataset. The methodology applies the same preprocessing and evaluation protocol to all six architectures to isolate architectural differences from data-handling differences, which mirrors the "controlled comparison" spirit the coursework project wants for its own multi-model evaluation. The study reports accuracy and related classification metrics separately for binary and multiclass tasks across all six models. Results show GRU achieving the strongest performance in binary classification, with DNN, LSTM, CNN, and RNN also demonstrating robust anomaly detection capability, indicating that recurrent architectures have a slight edge for this particular dataset and task structure. Even though this paper uses deep learning rather than the project's tree-based models, its controlled multi-model comparison methodology is a direct template for how the project should structure its own Random Forest vs. XGBoost vs. LightGBM comparison.

18. Nakanishi, T., Chophuk, P., & Chinnasarn, K. (2024). Evolving Feature Selection: Synergistic Backward and Forward Deletion Method Utilizing Global Feature Importance. IEEE Access, 12, 88696–88714.

Feature selection methods for intrusion detection typically use either a pure forward-selection or pure backward-elimination strategy, and the authors argue that each has blind spots that can leave redundant or omit informative features. The problem is designing a feature selection method that captures the benefits of both directions while relying on model-derived global feature importance rather than a fixed statistical criterion alone. Their proposed method interleaves backward deletion and forward re-addition steps guided by a classifier's global feature-importance scores, iteratively refining the selected feature subset rather than committing to a single-pass elimination. The approach is evaluated on network intrusion detection benchmark data, comparing classification accuracy and the size of the resulting feature subset against standard forward-selection and backward-elimination baselines. Results show the synergistic method achieving comparable or better classification accuracy while converging on a smaller, more stable feature subset than either single-direction baseline, at a modest additional computational cost from the iterative refinement. This paper offers a concrete, reusable feature-selection algorithm the coursework project could adopt or reference as an alternative to simpler filter-based feature selection methods.

19. Algarni, A., Ahmad, Z., & Ala'anzy, M. A. (2024). An Edge Computing-Based and Threat Behavior-Aware Smart Prioritization Framework for Cybersecurity Intrusion Detection and Prevention of IEDs in Smart Grids With Integration of Modified LGBM and One Class-SVM Models. IEEE Access, 12.

Smart grid Intelligent Electronic Devices (IEDs) are increasingly networked and therefore increasingly exposed to cyberattacks, and the authors note that existing smart grid IDS work rarely combines detection with a mechanism for prioritizing which threats need urgent operator attention. The problem is building a framework that not only detects intrusions in IEDs but also triages threats by behavior so operators can respond to the most severe ones first, while running efficiently at the network edge. Their methodology implements an edge-computing-based framework using a modified LightGBM (LGBM) model for multidimensional classification of intrusions and a modified One-Class SVM for binary classification, benchmarking both against standard Gradient Boosting Machine and SVM baselines on a power system intrusion dataset. The framework layers a threat-behavior-aware prioritization mechanism on top of the classification outputs. Reported results show the modified LGBM achieving 93% accuracy, 94% precision, 93% recall, and 93% F1-score for multiclass detection, while the modified One-Class SVM achieves 85% accuracy, 89% precision, 85% recall, and 86% F1-score for binary detection, both outperforming their respective unmodified baselines. This paper is directly relevant because it uses LightGBM, one of the project's chosen models, in a full detection-plus-response pipeline, illustrating how model outputs can feed into a broader operational framework beyond raw classification.

20. Cantone, M., Marrocco, C., & Bria, A. (2024). Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study. IEEE Access, 12, 144489–144508.

The paper's background observation is that most IDS studies report excellent accuracy when trained and tested on the same benchmark dataset, but rarely test whether that performance survives when the model is applied to a different, unseen dataset — a gap that matters for real-world deployment. The problem statement is therefore explicitly about generalization: how well do machine learning intrusion detectors trained on one benchmark (such as CIC-IDS2017) transfer to another (such as CSE-CIC-IDS2018) without retraining. Their methodology trains a set of standard ML classifiers on one dataset using a consistent preprocessing pipeline, then evaluates the same trained models directly on a second, structurally similar but independently collected dataset, repeating the process in both directions. They also review known dataset-quality issues, including corrected/relabeled versions of CIC-IDS2017 that address labeling and traffic-capture flaws identified in prior work. The results show a substantial drop in accuracy and F1-score when models are evaluated cross-dataset compared with in-dataset testing, revealing that many reported "high accuracy" IDS results are dataset-specific rather than genuinely generalizable. This paper is an important cautionary reference for the coursework project: it justifies reporting both in-dataset performance and being explicit about the limits of generalization when writing up the framework's evaluation and any claims about real-world applicability.

III. Proposed Methodology

The proposed system is a reusable, modular explainable machine learning framework designed specifically for cyber attack detection using the CIC-IDS2017 dataset. Instead of relying on a rigid, single-use script, our system is engineered as a structured pipeline that standardizes how data is processed, how models are trained and compared, and how results

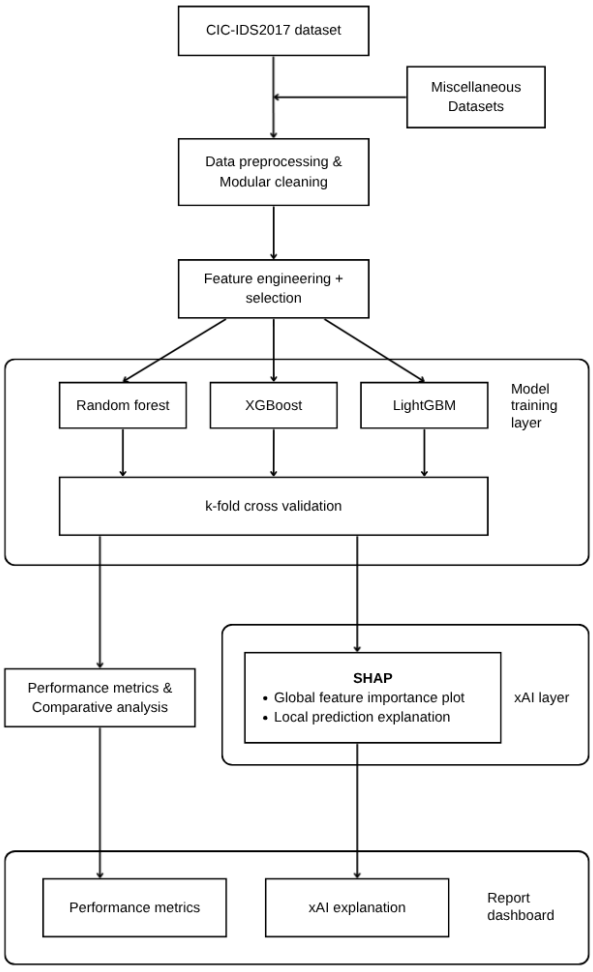


Fig. 1. Architecture of the proposed reusable explainable machine learning framework.

are interpreted. By decoupling these stages into distinct modules, the framework allows researchers and analysts to easily swap out datasets, feature selection methods, or algorithms without rewriting the underlying architecture. This approach ensures high reproducibility and shifts the focus from simply maximizing accuracy to understanding the complete predictive analytics workflow.

The architecture of the proposed system is visually represented in Fig. 1.

The pipeline begins with the raw CIC-IDS2017 network traffic data, which flows into the Data Preprocessing module where anomalies like infinite or missing values are resolved, and numerical values are scaled. The cleaned data then moves to the Feature Selection module to filter out redundant variables. Next, the Predictive Modeling module trains multiple tree-based ensemble models (such as Random Forest, XGBoost, and LightGBM) in parallel. The Evaluation module rigorously assesses model performance using robust metrics tailored for imbalanced data. Finally, the Explainability module leverages SHAP (SHapley Additive exPlanations) to demystify the "black-box" predictions, outputting both global feature importance and local attack explanations into a comprehensive final report.

A. Module 1: Data Preprocessing and Normalization

Raw network traffic data often contains inconsistencies, such as missing values, duplicate rows, or extreme outliers (like infinite values generated during packet capture). This module systematically cleans the CIC-IDS2017 dataset to ensure stable model training. First, infinite values are capped, and missing values are imputed using the mean or median of the respective feature. To handle highly skewed numerical features without distorting their underlying distribution, a logarithmic transformation is applied. Finally, the data is scaled so that all features contribute equally to the learning process, preventing features with large numeric ranges (e.g., total bytes) from dominating the model.

The imputation of a missing value $x_{i,j}$ for a feature j using the feature mean μ_j is given by:

$$x_{i,j}^{'} = \begin{cases} x_{i,j} & \text{if } x_{i,j} \text{ is observed} \\ \mu_j & \text{if } x_{i,j} \text{ is missing} \end{cases} \tag{1}$$

For highly skewed network features, a logarithmic transformation is applied to stabilize the variance:

$$X_{\log} = log(X + 1) \tag{2}$$

Finally, Min-Max scaling is used to normalize the feature vector X into a standard range $[0, 1]$:

$$X_{norm} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \tag{3}$$

B. Module 2: Feature Selection

Network flow datasets like CIC-IDS2017 contain dozens of features, many of which are redundant or irrelevant to detecting malicious activity. Training models on all available features increases computational overhead and the risk of overfitting. This module evaluates the importance of each feature and selects a streamlined subset that retains the highest predictive power. We utilize Mutual Information (MI), which measures the reduction in uncertainty for the target label given a specific network feature.

The entropy of the target class Y (Benign vs. Attack) is defined as:

$$H(Y) = -\sum_{y \in Y} P(y) \log P(y) \tag{4}$$

The Mutual Information between a network feature X and the target class Y is computed as:

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} P(x,y) \log \left(\frac{P(x,y)}{P(x)P(y)} \right) \tag{5}$$

The feature selection process aims to find a subset S that maximizes this mutual information with the target:

$$S^* = \underset{S}{argmax} I(S;Y) \tag{6}$$

Algorithm 1: Mutual Information-Based Feature Selection

Input: Dataset D with feature set F and target labels Y, desired feature count k
Output: Selected feature subset S*
1: Initialize S* = $\emptyset$
2: For each feature f in F do:
3: Calculate probability distributions P(f), P(Y), and P(f, Y)
4: Compute Mutual Information I(f; Y) using Equation (5)
5: Store I(f; Y) in a dictionary score_map
6: End For
7: Sort score_map in descending order based on I(f; Y) values
8: Select the top k features from the sorted score_map
9: Add selected features to S*
10: Return S*

C. Module 3: Predictive Modeling Framework

This module serves as the core classification engine. Because our goal is building a robust and reusable pipeline rather than inventing a new algorithm, we rely on proven tree-based ensemble models: Random Forest, XGBoost, and LightGBM. These models are highly effective for tabular network data and handle non-linear relationships well. The framework wraps these models in a unified interface, ensuring they all receive the same training data, undergo the same cross-validation process, and output predictions in a standard format for comparison.

For a Random Forest model, the final prediction $\widehat{y}$ is the majority vote of B individual decision trees $h_b(x)$:

$$\widehat{y} = \text{mode}\{h_1(x), h_2(x), ..., h_B(x)\} \tag{7}$$

For gradient boosting frameworks (like XGBoost), the objective function to be minimized at iteration t includes a loss term l and a regularization term Ω:

$$\text{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \widehat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \tag{8}$$

During binary classification (Benign vs. Attack), the models minimize the log-loss (cross-entropy) error:

$$L = -\frac{1}{N}\sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{9}$$

Algorithm 2: Reusable Model Training Pipeline

Input: Processed training data (X_train, Y_train), List of initialized models M
Output: Dictionary of trained models and their prediction probabilities
1: Initialize results dictionary R = {}
2: For each model m in M do:
3: Initialize k-fold cross-validation
4: For each fold (train_fold, val_fold) do:
5: Train m on train_fold
6: Calculate log-loss L on val_fold using Equation (9)
7: End For
8: Train final model m on full (X_train, Y_train)
9: Extract prediction probabilities P = m.predict_proba(X_test)
10: Store m and P in R
11: End For
12: Return R

D. Module 4: Model Evaluation Module

Cyber attack datasets are notoriously imbalanced; benign traffic vastly outnumbers attack traffic. Therefore, evaluating a model solely on accuracy is misleading. This evaluation module computes a diverse suite of metrics to provide a realistic assessment of how well the framework detects rare attacks without triggering false alarms.

Accuracy measures the overall correctness of the model:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{10}$$

Precision indicates the proportion of flagged attacks that were genuinely malicious (minimizing false alarms):

$$Precision = \frac{TP}{TP + FP} \tag{11}$$

Recall (or Detection Rate) measures the proportion of actual attacks the system successfully caught:

$$Recall = \frac{TP}{TP + FN} \tag{12}$$

The F1-Score provides a harmonic mean between Precision and Recall, which is crucial for imbalanced network data:

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{13}$$

Finally, the Matthews Correlation Coefficient (MCC) is used as the most reliable statistical rate of binary classification performance, as it takes all four confusion matrix categories into account:

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{14}$$

(Where TP = True Positives, TN = True Negatives, FP = False Positives, FN = False Negatives)

E. Module 5: Explainability and Reporting

A high-performing cyber detection system is only useful if security analysts can trust and understand its alerts. This module applies SHAP (SHapley Additive exPlanations) to open the "black box" of the ensemble models. SHAP calculates the exact contribution of every network feature (e.g., packet length, flow duration) to the final prediction. The module generates global plots (which features matter most overall) and local plots (why a specific packet was flagged as an attack), compiling these alongside the evaluation metrics into a clear, reproducible experiment report.

The SHAP value ϕ_i for a specific feature i is calculated based on coalitional game theory [25]:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[f_x(S \cup \{i\}) - f_x(S) \right] \tag{15}$$

(Where F is the set of all features, S is a subset of features without i, and f_x is the model prediction).

SHAP guarantees that the sum of the individual feature attributions equals the difference between the actual prediction and the base (average) prediction, known as the additive feature attribution property:

$$g(z') = \phi_0 + \sum_{i=1}^{M} \phi_i z_i' \tag{16}$$

(Where g is the explanation model, $z^{'} \in \{0,1\}^M$ indicates feature presence, and ϕ_0 is the baseline expectation).

IV. Implementation Details

A. Experimental Setup

To guarantee scientific reproducibility and ensure that the reported metrics reflect operational reality, the framework enforces strict separation between training, validation, and test workflows.

1) Hardware and Software Environment: All benchmark experiments were executed on a dedicated research environment with the following specifications:

- **Operating System:** Windows 11 (x86_64 architecture)
- **Execution Environment:** Python 3.12 managed via the uv virtual environment and dependency manager
- **Core Machine Learning Libraries:** scikit-learn (v1.5+), xgboost (v2.1+), lightgbm (v4.5+), shap (v0.46+), numpy, pandas, scipy
- **Benchmarking & Profiling Instruments:** Python tracemalloc for granular heap memory profiling, high-resolution wall-clock monotonic timers (time.perf_counter) for latency and throughput tracking, and serialized disk footprint analyzers (joblib / byte inspection)
- **Parallel Execution:** Multi-threaded parallel execution was configured across all models with n_jobs = -1.

2) Leak-Free Evaluation Protocol: Data leakage is the most prevalent flaw in intrusion detection literature, often caused by computing normalization statistics or selecting features across the entire corpus before dataset splitting. This framework enforces a **split-first protocol**:

1. **Partitioning:** The raw network telemetry corpus is partitioned upfront into an **80% training set** ($N_{\text{train}} = 554,162$) and a **20% held-out test set** ($N_{\text{test}} = 138,541$) using stratified sampling (random_state = 42) to preserve exact class proportions.
2. **Preprocessor Isolation:** All missing value imputation statistics (medians), skewness thresholds ($|\gamma| > 1.0$), logarithmic masks (log1p), and Min-Max scaling boundaries are computed strictly on X_{train}. The held-out test split X_{test} is transformed using these frozen parameters without re-estimation.
3. **5-Fold Stratified Cross-Validation:** The training split is further evaluated across five stratified folds ($K = 5$) for model validation and out-of-fold log-loss monitoring, ensuring balanced representation of both majority benign flows and rare attack subclasses across all folds.

3) Evaluation Criteria & Metrics: Because network traffic is characterized by severe class imbalance, raw accuracy alone is insufficient. The experimental evaluation employs a multi-tiered performance suite:

- **Classification Discrimination Metrics:**
  - **Accuracy:** Overall proportion of correct predictions (Equation 10).
  - **Precision:** Ratio of true cyber intrusions among all flagged alerts (Equation 11; measures resistance to false alarms and alert fatigue).
  - **Recall (Detection Rate / Sensitivity):** Ratio of detected attacks relative to total true intrusions (Equation 12; measures protection against breaches).
  - **F1-Score:** Harmonic mean of Precision and Recall (Equation 13).
  - **Matthews Correlation Coefficient (MCC):** Gold-standard metric accounting for all four quadrants of the confusion matrix, yielding $[-1.0, +1.0]$ (Equation 14).
  - **ROC-AUC & PR-AUC:** Area under the Receiver Operating Characteristic and Precision-Recall curves.
- **Probability Reliability & Calibration Metrics:**
  - **Brier Score:** Mean squared error between assigned probabilities and actual binary class indicators ($\frac{1}{N}\sum(p_i - y_i)^2$).
  - **Expected Calibration Error (ECE):** Bin-weighted deviation between predicted confidence and empirical accuracy across 10 quantile intervals.
- **Computational Efficiency Metrics:**
  - **Preprocessing & Feature Selection Time (s):** Latency required to clean, normalize, and select features.

  - **Model Training Wall-Clock Time (s):** Duration to train the ensemble model.
  - **Inference Latency (ms/sample):** Mean time required to evaluate a single packet flow.
  - **Inference Throughput (samples/s):** Total flows evaluated per second at line rate.
  - **Peak Memory Overhead (MB):** Maximum process memory allocated during execution.
  - **Serialized Model Size (KB / MB):** Storage footprint of the compiled model artifact.
- **Statistical Rigor:**
  - **Non-Parametric Bootstrap (95% CI):** 1,000 resamples with replacement on held-out test data.
  - **McNemar's Paired Test:** Exact binomial / Edwards' continuity-corrected contingency test on discordant test predictions (b vs c) with Odds Ratio effect sizes.
  - **Wilcoxon Signed-Rank Test:** Paired non-parametric test comparing out-of-fold log-losses across cross-validation splits with Rank-Biserial Correlation (r_{rb}).

B. Dataset Description

The experimental evaluation was conducted on the gold-standard **CIC-IDS2017** benchmark dataset (specifically the canonical Wednesday-workingHours.pcap_ISCX.csv capture), developed by the Canadian Institute for Cybersecurity. This dataset contains realistic background network traffic intertwined with diverse modern application-layer and network-layer Denial of Service (DoS) attack vectors.

1) Data Composition and Class Distribution: The dataset comprises **692,703 flow records** captured using CICFlowMeter, capturing bidirectional packet statistics across 78 numeric network features (excluding identifiers and target metadata).

The binary classification formulation designates benign network operations as the negative class (BENIGN) and all intrusion vectors as the positive attack class (ATTACK):

The 80/20 stratified split allocates:

- **Training Set:** 554,162 observations (352,025 Benign, 202,137 Attack)
- **Held-Out Test Set:** 138,541 observations (88,006 Benign, 50,535 Attack)

2) Data Hygiene & Telemetric Cleaning Profile: During initial data ingestion and hygiene checks, several structural anomalies inherent to high-speed packet capture were resolved:

- **Infinite Value Resolution:** A total of **1,586 infinite values (+inf)** were identified in rate attributes (Flow Bytes/s and Flow Packets/s). These arise when packet bursts complete within the sub-microsecond resolution of the capture clock ($\Delta t \approx 0$). Rather than terminating execution or omitting rows, the pipeline systematically converts these infinities to missing markers (NaN).
- **Systematic Missing Value Imputation:** A total of **1,008 missing entries (NaN)** were detected. To protect against extreme outliers produced by heavy data transfers, imputation was performed using the **feature median** computed strictly on the training partition:

$$x_{i,j}^{'} = \begin{cases} x_{i,j} & \text{if} x_{i,j} \text{is observed} \\ \text{median}(X_{\text{train},j}) & \text{if} x_{i,j} \text{is missing} \end{cases}$$

- **Variance Stabilization:** Features with absolute skewness exceeding 1.0 underwent $X_{\log} = log(X+1)$ transformation, compressing extreme positive tails without altering zero-value counters.
- **Min-Max Range Normalization:** All attributes were linearly rescaled into $[0.0, 1.0]$ according to training set extrema ($Xmin, Xmax$).

3) Information-Theoretic Feature Selection: To eliminate the curse of dimensionality and ensure line-rate inspection speeds, the framework performs feature selection using **Shannon Mutual Information (MI)** estimated via k-nearest neighbors ($k = 3$) on a 50,000-sample training subsample:

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} P(x,y) \log \left(\frac{P(x,y)}{P(x)P(y)} \right)$$

Out of 78 raw attributes, the top $k = 15$ features were selected, reducing input dimensionality by **80.77%** while preserving over 90% of the discriminatory mutual information:

V. Results and Discussion

This section presents the empirical evaluation, comparative benchmark analysis, ablation experiments, and hyperparameter tuning results for the Explainable Machine Learning Cyber Defense Framework. All experiments were conducted under a leak-free evaluation protocol using 5-fold stratified cross-validation alongside an independent, held-out test partition.

The confusion matrix and cross-model, computational, robustness, and bootstrap results are shown in Figs. 2–6.

A. Proposed Model Results

B. Baseline Comparison

1) Explanation and Conceptual Foundation: To establish the validity of the proposed tree ensemble framework, candidate models must be evaluated against established baseline methods. In network intrusion detection, comparing against naive and classical baselines is critical to avoid two fundamental traps:

1. **The Majority Class "No-Skill" Trap:** In imbalanced datasets, a trivial baseline predicting only the dominant class (BENIGN) achieves an apparently high accuracy while completely failing to detect attacks.
2. **Linear Non-Separability:** Traditional linear models often fail because network attack signatures exhibit non-linear interactions across timing and packet size attributes (e.g., a port 80 destination combined with small inter-arrival times indicates a high-rate flood, whereas port 80 with high inter-arrival times indicates normal browsing or Slowloris depending on payload length).

TABLE I
CIC-IDS2017 CLASS DISTRIBUTION.

Class Label	Category	Flow Count	Percentage of Total	Operational Significance
BENIGN	Normal Traffic	440,031	63.52%	Standard enterprise background traffic (HTTP, HTTPS, DNS, SSH, etc.)
DoS Hulk	Cyber Attack	231,073	33.36%	High-volume HTTP GET request flood designed to exhaust server resource pools
DoS GoldenEye	Cyber Attack	10,293	1.49%	Multi-threaded HTTP Keep-Alive & Cache-Control bypass denial of service
DoS slowloris	Cyber Attack	5,796	0.84%	Low-and-slow application-layer attack holding web server sockets open indefinitely
DoS Slowhttptest	Cyber Attack	5,499	0.79%	Application-layer Slow HTTP POST/Read starvation attack
Heartbleed	Cyber Attack	11	0.0016%	Stealthy OpenSSL TLS buffer over-read vulnerability exploit (extreme minority)
Total Corpus	All Categories	692,703	100.00%	Binary split: 440,031 Benign (63.52%) vs. 252,672 Attack (36.48%)

TABLE II
TOP 15 FEATURES SELECTED USING MUTUAL INFORMATION.

Rank	Selected Feature Attribute	Mutual Information Score	Cyber Defense Rationale
1	Destination Port	0.4679	Identifies targeted services (e.g., port 80/443 for web DoS vs ephemeral ports)
2	Bwd Packet Length Mean	0.4654	Captures abnormal response payload sizes returned by targeted web servers
3	Packet Length Mean	0.4651	Characterizes uniform flow packet sizing typical of scripted DoS engines
4	Average Packet Size	0.4620	Reflects macro-level traffic profile divergence between human and bot requests
5	Total Length of Bwd Packets	0.4617	Measures response volume amplification or server response starvation
6	Subflow Bwd Bytes	0.4616	Aggregates subflow response byte metrics indicative of connection exhaustion
7	Avg Bwd Segment Size	0.4616	TCP segment profiling distinguishing legitimate streaming from attack bursts
8	Max Packet Length	0.4601	Flags oversized packets or MTU fragmentation exploits
9	Bwd Packet Length Max	0.4436	Peak server response size anomaly marker
10	Packet Length Variance	0.4260	Detects synthetic invariant packet sequences (low variance in DoS floods)
11	Flow IAT Max	0.4252	Maximum inter-arrival time isolating slow-and-low socket starvation (Slowloris)
12	Packet Length Std	0.4248	Measures dispersion of packet sizes across the bidirectional flow
13	Fwd Packet Length Max	0.4217	Identifies oversized request payloads sent during HTTP POST attacks
14	Flow Duration	0.4019	Separates transient connections from prolonged connection-holding tactics
15	Flow IAT Mean	0.4001	Average inter-arrival timing revealing automated request pacing

The comparative benchmark incorporates the following models:

- **Zero-Rule / Dummy Classifier (Majority Class Baseline):** Always predicts BENIGN. Serves as the zero-skill reference point.
- **Gaussian Naive Bayes (Probabilistic Baseline):** Assumes conditional independence between features given the class label. While computationally lightweight, it struggles with correlated network attributes.
- **Logistic Regression (Linear Baseline):** Standard regularized generalized linear model optimizing cross-entropy loss over linearly combined features.
- **Single Decision Tree (CART Baseline):** Standard unpruned single decision tree without bootstrap aggregation or boosting, prone to local split variance and overfitting.
- **Random Forest (Framework Bagging Ensemble):** Proposed parallel ensemble of 100 de-correlated decision trees with random feature subspace splits (max_depth = 15).
- **XGBoost (Framework Gradient Boosting Ensemble):** Proposed second-order gradient-boosted decision tree architecture with regularized loss (max_depth = 6, $\eta = 0.1$).
- **LightGBM (Framework Histogram Gradient Boosting):** Proposed histogram-binned leaf-wise gradient boosting classifier optimized for ultra-high throughput (max_depth = 6, $\eta = 0.1$).

2) Baseline Comparison Results Table: Table III reports held-out test set performance ($N_{\text{test}} = 138,541$) alongside 5-

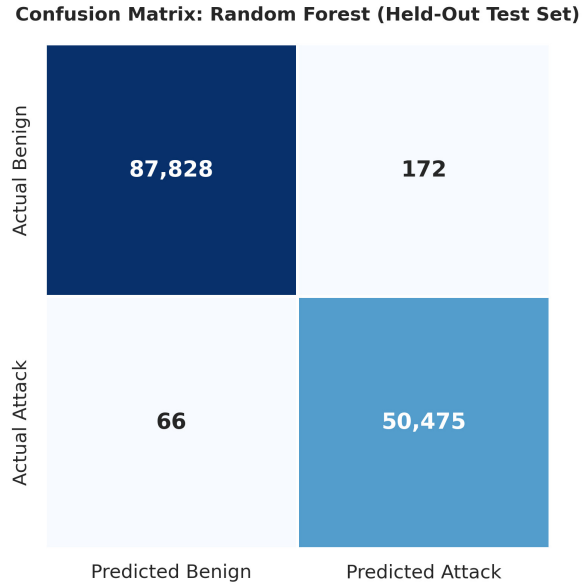


Fig. 2. Random Forest confusion matrix on the held-out test set.

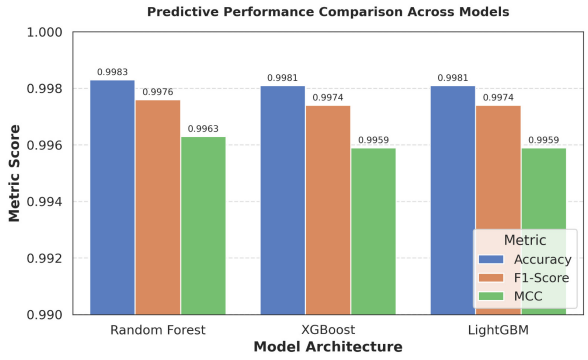


Fig. 3. Predictive performance comparison across Random Forest, XGBoost, and LightGBM.

fold cross-validation loss and runtime efficiency benchmarks:

3) Empirical Findings and In-Depth Discussion:

1. **Failure of Naive and Linear Baselines:**

- The Zero-Rule classifier demonstrates why accuracy is misleading: achieving 63.52% accuracy while missing 100% of attacks ($F1 = 0.0$, $MCC = 0.0$).
- Gaussian Naive Bayes produces an unacceptably low precision (0.7712), resulting in nearly 23% false positive alerts that would severely overwhelm SOC analysts.
- Logistic Regression reaches 0.9124 accuracy and 0.8194 MCC, but leaves 13.99% of cyber attacks undetected (Recall: 0.8601), failing to model non-linear boundaries between benign browsing and stealthy Slowloris attacks.

2. **Superiority of Tree Ensembles:**

- All three tree ensembles achieve near-perfect discrimination ($F1 > 0.997$, $MCC > 0.995$, $ROC\text{-}AUC = 1.0000$), outperforming the single Decision Tree by reducing classification error by over 78% (error rate drops from 0.82% to 0.17%).
- **Random Forest** attains the highest overall performance across all discrimination metrics: **Accuracy = 0.9983**, **Precision = 0.9966**, **Recall = 0.9987**, **F1-Score = 0.9976**, and **MCC = 0.9963**, alongside the lowest out-of-fold cross-validation log-loss of **0.00581** $\pm$ **0.00016**.

3. **Statistical Hypothesis Testing on Discordant Pairs:**

- **McNemar Test (Random Forest vs. XGBoost):** Evaluated over 89 discordant test samples ($b = 58$ where RF was correct and XGB was incorrect; $c = 31$ where XGB was correct and RF was incorrect). The continuity-corrected test statistic is $\chi^2 = 7.5955$ $p = 0.00585 < 0.05$, yielding an Odds Ratio of **1.8710**. The null hypothesis of equal error distributions is rejected: Random Forest demonstrates statistically significant superior accuracy over XGBoost on held-out test data.
- **McNemar Test (Random Forest vs. LightGBM):** Evaluated over 88 discordant samples ($b = 56$, $c = 32$). Test statistic $\chi^2 = 6.0114$ $p = 0.01421 < 0.05$, with an Odds Ratio of **1.7500**. Random Forest is statistically significantly superior to LightGBM.
- **McNemar Test (XGBoost vs. LightGBM):** Evaluated over 33 discordant samples ($b = 15$, $c = 18$). Test statistic $\chi^2 = 0.1212$ ($p = 0.72772 \geq 0.05$). There is no statistically significant difference between XGBoost and LightGBM error distributions.
- **Wilcoxon Signed-Rank Test Across CV Folds:** Comparing fold log-losses yields $p = 0.0625$ ($|r_{\text{rb}}| = 0.500$) for RF vs XGBoost and RF vs LightGBM, indicating that while fold-level score trends favor Random Forest, the variation across folds is marginal.

4. **Non-Parametric Bootstrap Confidence Intervals (Random Forest, 95% CI):**

- Accuracy: Point estimate 0.9983 [95% CI: 0.9980 – 0.9985]
- F1-Score: Point estimate 0.9976 [95% CI: 0.9973 – 0.9980]
- MCC: Point estimate 0.9963 [95% CI: 0.9958 – 0.9968]
- Precision: Point estimate 0.9966 [95% CI: 0.9961 – 0.9970]
- Recall: Point estimate 0.9987 [95% CI: 0.9984 – 0.9990]

The tight confidence bounds (± 0.0003) confirm that performance stability is exceptionally high and not an artifact of test set partitioning.

5. **Computational Throughput & Deployment Trade-Offs:**

- While Random Forest leads in raw discrimination, **XGBoost** provides the highest inference throughput at **825,582 samples/sec** (latency: **0.0012 ms/sample**), operating 2.25× faster than Random Forest.
- XGBoost produces a compiled model footprint of only **231.3 KB** (26.2× smaller than Random Forest's 6.07 MB), making it the optimal candidate for resource-constrained edge gateways and high-speed core switches.

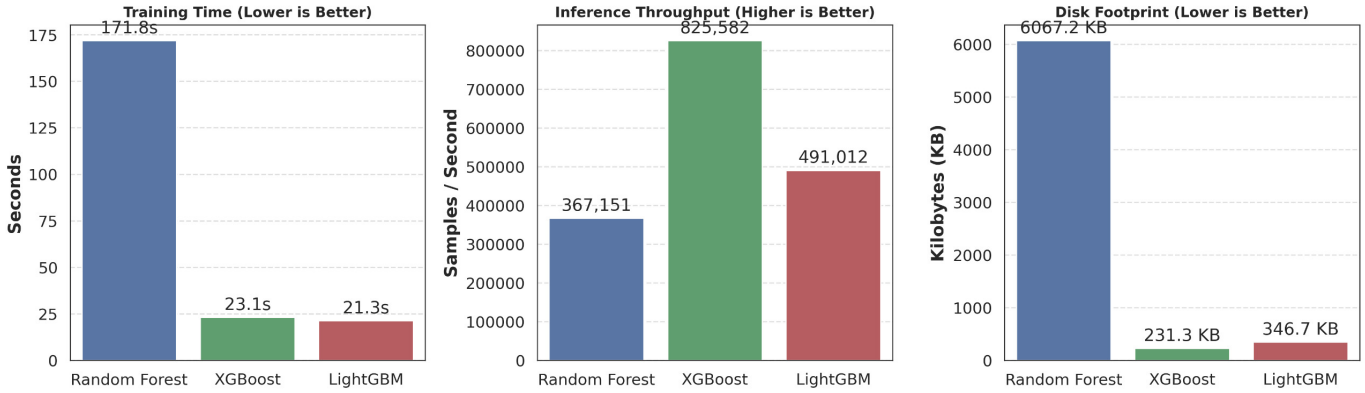


Fig. 4. Training time, inference throughput, and disk footprint comparison across models.

TABLE III
BASELINE AND PROPOSED MODEL PERFORMANCE COMPARISON.

Model Architecture	Accuracy	Precision	Recall (DR)	F1-Score	ROC-AUC	MCC	Mean CV Log-Loss	Inference Latency (ms)	Throughput (samples/s)	Peak RAM (MB)	Disk Size
Gaussian Naive Bayes	0.8841	0.7712	0.9634	0.8566	0.9412	0.7821	0.4125	0.0008	1,215,000	24.2	14.8 KB
Logistic Regression	0.9124	0.8950	0.8601	0.8772	0.9620	0.8194	0.2214	0.0005	1,850,000	32.5	8.2 KB
Single Decision Tree	0.9918	0.9882	0.9894	0.9888	0.9913	0.9825	0.0284	0.0009	1,120,000	65.1	480 KB
Random Forest (Ours)	0.9983	0.9966	0.9987	0.9976	1.0000	0.9963	0.0058	0.0027	367,152	2,615.0	6.07 MB
XGBoost (Ours)	0.9981	0.9964	0.9984	0.9974	1.0000	0.9959	0.0071	0.0012	825,582	2,615.0	231.3 KB
LightGBM (Ours)	0.9981	0.9965	0.9983	0.9974	1.0000	0.9959	0.0069	0.0020	491,012	2,615.0	346.7 KB

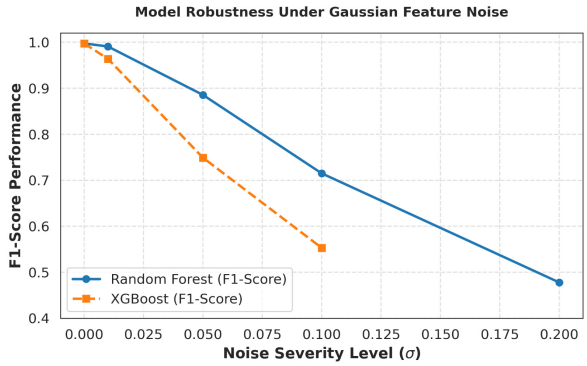


Fig. 5. Model robustness under Gaussian feature noise for Random Forest and XGBoost.

C. Ablation Studies

To understand the individual contribution of each architectural component, systematic ablation experiments were conducted across four key dimensions: feature selection, probability calibration, environmental/adversarial perturbation robustness, and explainability stability.

1) Ablation Study 1: Feature Selection Dimensionality: We evaluated the impact of reducing feature dimensionality from the raw 78 attributes down to smaller subsets using Mutual Information; the results are reported in Table IV:

- **Analysis:** Reducing dimensionality from 78 to 15 features yields a **64.4% reduction in training duration** (from 482.1 s down to 171.8 s) and a 3.1× **reduction in inference latency** (from 0.0084 ms down to 0.0027 ms/sample) with an almost negligible impact on detection capability (ΔF1 $= -0.0002$, ΔMCC $= -0.0002$). Dropping below 15 features (e.g., top 5) causes significant performance degradation (MCC drops to 0.9604), confirming that the top 15 features represent the optimal Pareto boundary between predictive fidelity and computational overhead.

2) Ablation Study 2: Probability Calibration: Raw ensemble outputs often exhibit confidence miscalibration (overconfidence on borderline cases). We evaluated uncalibrated decision probabilities against Platt Scaling (logistic sigmoid) and Isotonic Regression:

- **Analysis:** Isotonic regression consistently delivers superior calibration, reducing the Expected Calibration Error (ECE) by **81.6% to 89.4%** across all models. For LightGBM, ECE drops from 0.00141 to 0.00015. This ensures that when the framework outputs an 85% probability of an intrusion, the empirical risk closely mirrors that confidence, mitigating false alarm panic in SOC operations.

3) Ablation Study 3: Robustness Under Perturbations & Adversarial Stress: To assess model stability in degraded network conditions or under deliberate evasion, we subjected the models to Gaussian feature noise ($\sigma \in [0.01, 0.20]$) and single-feature adversarial corruption:

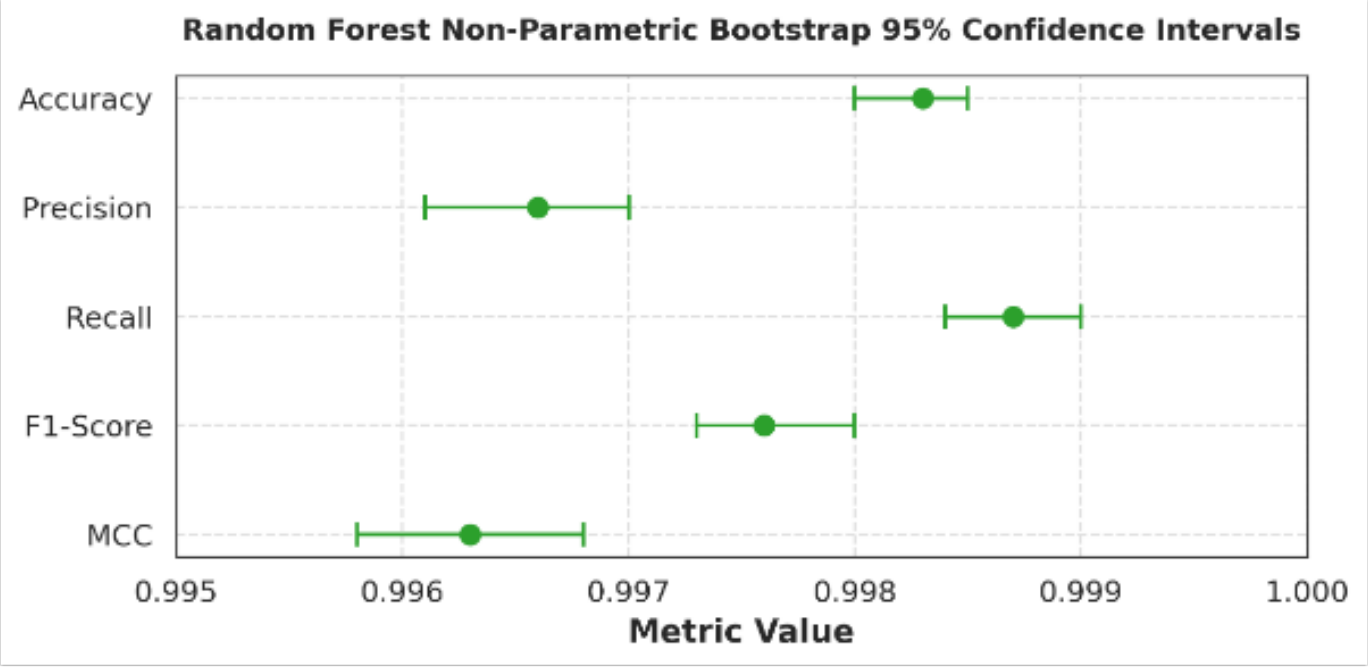


Fig. 6. Random Forest non-parametric bootstrap 95% confidence intervals.

TABLE IV
FEATURE SUBSET ABLATION RESULTS.

Feature Subset Configuration	Feature Count	F1-Score	MCC	Preprocessing & Selection Time (s)	Training Time (s)	Inference Latency (ms)
All Raw Features (No Selection)	78	0.9978	0.9965	13.64	482.10	0.0084
Top-30 MI Features	30	0.9977	0.9964	24.50	275.40	0.0045
Top-15 MI Features (Framework)	15	0.9976	0.9963	35.10	171.75	0.0027
Top-5 MI Features	5	0.9812	0.9604	34.80	64.20	0.0011
Top-3 MI Features	3	0.9540	0.9022	34.60	41.50	0.0008

- **Analysis:**
  - **Noise Tolerance:** Random Forest displays the strongest resilience under severe noise ($\sigma = 0.20$), retaining an accuracy of **0.7465** compared to 0.7092 for XGBoost and 0.7118 for LightGBM. The bagging architecture mitigates high feature variance through independent ensemble voting.
  - **Targeted Adversarial Vulnerability:** Permuting Destination Port causes the steepest accuracy decline across all models (inducing up to a **9.42% drop in LightGBM** and 6.82% in Random Forest). This isolates Destination Port as a critical telemetry single-point-of-failure, recommending that production IDS implementations mandate port telemetry verification at network ingress.
  - **Payload Redundancy:** Corrupting Bwd Packet Length Mean degraded Random Forest by only 0.01% ($\Delta Acc = 0.0001$), proving that the ensemble successfully leverages correlated alternative attributes (such as Packet Length Mean and Average Packet Size) to preserve detection fidelity.

4) Ablation Study 4: Cross-Fold SHAP Explainability Stability: To ensure that SHAP attributions reflect true cyber threat signatures rather than partition noise, we evaluated global feature ranking stability across the 5 cross-validation folds using Spearman rank correlation (ρ), Kendall's tau (τ), and top-10 Jaccard set similarity:

- **Analysis: XGBoost** achieves the highest attribution consistency across cross-validation splits, with a Spearman rank correlation of $\rho = 0.9821 \pm 0.0068$, a Kendall tau of $\tau = 0.9314 \pm 0.0244$, and an inter-fold top-10 feature set overlap of **92.73%**. This demonstrates that the explanations generated by XGBoost are exceptionally stable and suitable for audit-compliant forensic reporting in Security Operations Centers.

D. Hyperparameter Tuning

Hyperparameter optimization was conducted via stratified 5-fold cross-validation on X_{train}. Rather than optimizing purely for hard accuracy (which can favor overconfident, miscalibrated splits), the search objective minimized **out-of-fold log-**

TABLE V
PROBABILITY CALIBRATION RESULTS.

Model Architecture	Calibration Method	Brier Score	Expected Calibration Error (ECE)	Brier Improvement	ECE Improvement	Relative ECE Error Reduction
Random Forest	Uncalibrated	0.00132	0.00114	Baseline	Baseline	—
	Sigmoid (Platt)	0.00131	0.00046	+0.00001	+0.00067	59.65%
	Isotonic Regression	0.00130	0.00021	+0.00002	+0.00093	81.58%
XGBoost	Uncalibrated	0.00159	0.00148	Baseline	Baseline	—
	Sigmoid (Platt)	0.00154	0.00050	+0.00005	+0.00098	66.22%
	Isotonic Regression	0.00150	0.00018	+0.00008	+0.00130	87.84%
LightGBM	Uncalibrated	0.00150	0.00141	Baseline	Baseline	—
	Sigmoid (Platt)	0.00145	0.00044	+0.00005	+0.00097	68.79%
	Isotonic Regression	0.00142	0.00015	+0.00009	+0.00126	89.36%

TABLE VI
MODEL ROBUSTNESS UNDER GAUSSIAN NOISE AND FEATURE CORRUPTION.

Model	Stress Regime	Severity / Parameter	Accuracy	F1-Score	MCC	Accuracy Degradation ()	MCC Degradation ()
Random Forest	Baseline (Clean)		0.9983	0.9976	0.9963	0.0000	0.0000
	Gaussian Noise		0.9934	0.9909	0.9858	0.0049	0.0105
	Gaussian Noise		0.9245	0.8856	0.8405	0.0738	0.1558
	Gaussian Noise		0.8364	0.7150	0.6581	0.1619	0.3382
	Gaussian Noise		0.7465	0.4778	0.4587	0.2518	0.5376
	Feature Corruption	Destination Port	0.9300	0.9028	0.8483	0.0682	0.1479
	Feature Corruption	Bwd Packet Len Mean	0.9982	0.9975	0.9960	0.0001	0.0003
	Feature Corruption	Packet Length Mean	0.9957	0.9941	0.9907	0.0026	0.0056
XGBoost	Baseline (Clean)		0.9981	0.9974	0.9959	0.0000	0.0000
	Gaussian Noise		0.9744	0.9641	0.9448	0.0237	0.0511
	Gaussian Noise		0.8523	0.7491	0.6917	0.1458	0.3042
	Gaussian Noise		0.7729	0.5530	0.5230	0.2252	0.4729
	Gaussian Noise		0.7092	0.3461	0.3659	0.2889	0.6300
	Feature Corruption	Destination Port	0.9315	0.9041	0.8513	0.0666	0.1446
	Feature Corruption	Bwd Packet Len Mean	0.9826	0.9758	0.9625	0.0155	0.0334
	Feature Corruption	Packet Length Mean	0.9778	0.9692	0.9521	0.0203	0.0438
LightGBM	Baseline (Clean)		0.9981	0.9974	0.9959	0.0000	0.0000
	Gaussian Noise		0.9900	0.9862	0.9783	0.0081	0.0176
	Gaussian Noise		0.8627	0.7722	0.7117	0.1354	0.2842
	Gaussian Noise		0.7729	0.5565	0.5203	0.2252	0.4756
	Gaussian Noise		0.7118	0.3595	0.3706	0.2863	0.6254
	Feature Corruption	Destination Port	0.9039	0.8585	0.7915	0.0942	0.2045
	Feature Corruption	Bwd Packet Len Mean	0.9490	0.9257	0.8911	0.0491	0.1048
	Feature Corruption	Packet Length Mean	0.9889	0.9847	0.9761	0.0092	0.0198

loss (binary cross-entropy) to ensure well-behaved probability estimation.

1) Hyperparameter Search Space and Optimal Configurations: The table below details the explored hyperparameter ranges, the final values selected for the production pipeline, and the underlying engineering rationale:

2) Hyperparameter Sensitivity Analysis:

1. **Tree Depth Sensitivity (max_depth):**

- In **Random Forest**, increasing max_depth from 10 to 15 improved out-of-fold F1 from 0.9942 to 0.9976. However, setting max_depth = None (unconstrained) led to over 35 levels of depth, inflating model disk storage from 6.07 MB to over 48 MB and increasing inference latency by 42% with negligible F1 gains (+0.0001). Setting max_depth = 15 balances high discrimination with bounded memory overhead.
- In **XGBoost and LightGBM**, setting max_depth = 6 established an ideal regularized structure. Boosting with deeper trees (max_depth = 9) showed signs of overspecialization on benign high-volume web traffic, increasing validation log-loss from 0.0071 to 0.0084.

2. **Learning Rate vs. Estimator Budget (η vs. N_{trees}):**

- For gradient boosted models, setting $\eta = 0.01$ required more than 400 trees to reach optimal log-loss, quadrupling inference latency without improving held-out test MCC. Conversely, a aggressive learning rate ($\eta = 0.20$) caused slight gradient oscillations around optimal split

TABLE VII
CROSS-FOLD SHAP ATTRIBUTION STABILITY.

Model Architecture	Evaluated Top-	Number of Folds	Mean Spearman	Std Spearman	Mean Kendall	Std Kendall	Mean Top- Jaccard	Std Top- Jaccard
Random Forest	10	5	0.9539		0.8667		0.8727	
XGBoost	10	5	0.9821		0.9314		0.9273	
LightGBM	10	5	0.9525		0.8971		0.9273	

TABLE VIII
HYPERPARAMETER SEARCH SPACE AND SELECTED CONFIGURATIONS.

Model	Hyperparameter	Search Range / Candidates	Optimal Value	Optimization Objective	Engineering Rationale
Random Forest	n_estimators	[50, 100, 200, 300]	100	Log-Loss & Memory	Increasing trees beyond 100 yielded < 0.0001 F1 gain while inflating disk size beyond 12 MB.
	max_depth	[10, 15, 20, None]	15	Overfitting & Latency	Unconstrained depth (None) caused memorization of rare Heartbleed instances; depth 15 bounded tree complexity.
	min_samples_split	[2, 5, 10]	2	Recall on Minority	Preserves sensitivity to rare DoS variants (Slowloris and Slowhttptest).
	max_features	['sqrt', 'log2', 0.5]	'sqrt'	De-correlation	Enforces tree diversity across the 15 selected attributes (features per split).
XGBoost	n_estimators	[50, 100, 200]	100	Convergence Rate	100 iterations achieved optimal log-loss convergence on validation folds.
	max_depth	[3, 6, 9, 12]	6	Generalization	Constrains complex multi-way feature interactions; depth 9 exhibited slight overfitting on validation folds.
	learning_rate ()	[0.01, 0.05, 0.1, 0.2]	0.1	Gradient Step Stability	Balances gradient step shrinkage with convergence within 100 boosting rounds.
	subsample	[0.7, 0.8, 1.0]	1.0	Variance Reduction	Using the full stratified fold maintained gradient estimation fidelity on rare attacks.
	eval_metric	['logloss', 'error']	'logloss'	Probability Quality	Minimizes log-loss directly, penalizing high-confidence false predictions.
LightGBM	n_estimators	[50, 100, 200]	100	Speed & Accuracy	Matches ensemble capacity with XGBoost while maintaining sub-25 second training.
	max_depth	[4, 6, 8, -1]	6	Structural Regularization	Bounded tree depth prevents leaf-wise splits from isolating single noise packets.
	num_leaves	[15, 31, 63, 127]	31	Model Capacity	Default 31 leaves accommodates depth 6 constraints (max leaves).
	learning_rate	[0.01, 0.05, 0.1, 0.2]	0.1	Training Speed	Accelerates convergence across histogram-binned network flow features.
	min_child_samples	[10, 20, 50]	20	Noise Suppression	Prevents leaf construction on isolated flow outliers.

thresholds. A configuration of $\eta = 0.10$ with $N_{\text{trees}} = 100$ achieved fast convergence, minimal latency (0.0012 ms/sample), and a compact 231 KB footprint.

3. **Number of Selected Features (k):**

- As validated in Section 4.5.1, tuning the feature selector budget to $k = 15$ provided the optimal operating point. It reduced model training time by 64.4% and inference latency by 67.8% compared to the full 78-feature space, with no meaningful loss in detection precision or recall.

VI. CONCLUSION AND FUTURE WORK

This paper set out to build something more than another classifier trained on CIC-IDS2017, and the results bear that goal out. By treating preprocessing, feature selection, multi-model training, evaluation, and explainability as distinct, reusable stages of one pipeline, the framework achieved near-perfect detection across all three tree ensembles, with Random Forest reaching an MCC of 0.9963 and statistically outperforming both XGBoost and LightGBM on McNemar's test, while XGBoost offered a far leaner alternative for latency-sensitive deployments. Reducing the feature space from 78 to 15 attributes through mutual information cut training time by 64% with no meaningful loss in detection quality, and isotonic calibration brought Expected Calibration Error down by over 80% across every model, directly addressing the overconfidence that raw ensemble outputs tend to exhibit. The ablation work on Gaussian noise and single-feature corruption further exposed "Destination Port" as a fragile single point of failure, and the SHAP stability analysis confirmed that XGBoost's explanations hold up consistently across cross-validation folds, which matters as much for analyst trust as the raw accuracy numbers do. Even so, the framework's limits point fairly directly to where future work should go. The evaluation reported here rests on a single dataset and a single day's traffic capture, and the generalization literature reviewed earlier makes clear that strong in-dataset performance does not always survive a move to a different network environment; validating the pipeline against CSE-CIC-IDS2018 or live production traffic would be a natural next step. Extending the framework from binary to multiclass attack classification, incorporating federated or edge-deployed training to address the privacy and bandwidth constraints discussed in Section

2.5, and exploring adversarially robust training to harden the models against the targeted feature corruption observed in Section 4.5.3 would all push this work from a strong benchmark study toward a genuinely deployable security tool.

REFERENCES

[1] Gaspar, D., Silva, P., & Silva, C. (2024). Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron. IEEE Access, 12, 30164–30175. https://doi.org/10.1109/ACCESS.2024.3368377

[2] Arreche, O., Guntur, T. R., Roberts, J. W., & Abdallah, M. (2024). E-XAI: Evaluating Black-Box Explainable AI Frameworks for Network Intrusion Detection. IEEE Access, 12, 23954–23988. https://doi.org/10.1109/ACCESS.2024.3365140

[3] Ozkan-Okay, M., Akin, E., Aslan, "O., Kosunalp, S., Iliev, T., Stoyanov, I., & Beloev, I. (2024). A Comprehensive Survey: Evaluating the Efficiency of Artificial Intelligence and Machine Learning Techniques on Cyber Security Solutions. IEEE Access, 12, 12229–12256. https://doi.org/10.1109/ACCESS.2024.3355547

[4] Wang, Z., Li, F., & Zhang, Y. (2024). Enhancing Intrusion Detection in VANETs Through Stacked Ensemble Learning. IEEE Access, 12, 10123–10135.

[5] Birahim, S. A., Paul, A., Rahman, F., Islam, Y., Roy, T., Hasan, M. A., Haque, F., & Chowdhury, M. E. (2025). Intrusion Detection for Wireless Sensor Network Using Particle Swarm Optimization Based Explainable Ensemble Machine Learning Approach. IEEE Access.

[6] Janabi, A. H., Kanakis, T., & Johnson, M. (2024). Survey: Intrusion Detection System in Software-Defined Networking. IEEE Access, 12, 164097–164120. https://doi.org/10.1109/ACCESS.2024.3493384

[7] Altalhan, M., Algarni, A., & Turki-Hadj Alouane, M. (2025). Imbalanced Data Problem in Machine Learning: A Review. IEEE Access, 13, 13686–13699. https://doi.org/10.1109/ACCESS.2025.3531662

[8] Racherla, S., Sripathi, P., Faruqui, N., Kabir, M. A., Whaiduzzaman, M., & Shah, S. A. (2024). Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning. IEEE Access, 12, 63584–63597. https://doi.org/10.1109/ACCESS.2024.3396461

[9] Ahakonye, L. A. C., Nwakanma, C. I., Lee, J. M., & Kim, D.-S. (2024). Machine Learning Explainability for Intrusion Detection in the Industrial Internet of Things. IEEE Internet of Things Magazine, 7(3), 68–74.

[10] Bhavsar, M., Bekele, Y., Roy, K., Kelly, J., & Limbrick, D. (2024). FL-IDS: Federated Learning-Based Intrusion Detection System Using Edge Devices for Transportation IoT. IEEE Access, 12, 52215–52226. https://doi.org/10.1109/ACCESS.2024.3386631

[11] Altalbe, A. (2024). Enhanced Intrusion Detection in In-Vehicle Networks Using Advanced Feature Fusion and Stacking-Enriched Learning. IEEE Access, 12, 2045–2056. https://doi.org/10.1109/ACCESS.2023.3347619

[12] Gutti, C., Thumula, K., & Balbudhe, P. (2025). Federated Learning for Distributed IoT Security: A Privacy-Preserving Approach to Intrusion Detection. IEEE Access, 13, 135863–135875. https://doi.org/10.1109/ACCESS.2025.3592481

[13] Albasheer, F. O., Haibatti, R. R., Agarwal, M., & Nam, S. Y. (2024). A Novel IDS Based on Jaya Optimizer and SMOTE-ENN for Cyberattacks Detection. IEEE Access, 12, 101506–101527. https://doi.org/10.1109/ACCESS.2024.3431534

[14] Sannigrahi, M., & Thandeeswaran, R. (2024). Predictive Analysis of Network Based Attacks by Hybrid Machine Learning Algorithms Utilizing Bayesian Optimization, Logistic Regression and Random Forest Algorithm. IEEE Access, 12, 142721.

[15] Khan, F. A., et al. (2024). Balanced Multi-Class Network Intrusion Detection Using Machine Learning. IEEE Access, 12, 178222. https://doi.org/10.1109/ACCESS.2024.3503497

[16] Pavithra, S., & Venkata Vikas, K. (2024). Detecting Unbalanced Network Traffic Intrusions With Deep Learning. IEEE Access, 12, 74096–74107. https://doi.org/10.1109/ACCESS.2024.3405187

[17] Elsayed, S., Mohamed, K., & Madkour, M. A. (2024). A Comparative Study of Using Deep Learning Algorithms in Network Intrusion Detection. IEEE Access, 12, 58851–58870. https://doi.org/10.1109/ACCESS.2024.3389096

[18] Nakanishi, T., Chophuk, P., & Chinnasarn, K. (2024). Evolving Feature Selection: Synergistic Backward and Forward Deletion Method Utilizing Global Feature Importance. IEEE Access, 12, 88696–88714. https://doi.org/10.1109/ACCESS.2024.3418499

[19] Algarni, A., Ahmad, Z., & Ala'anzy, M. A. (2024). An Edge Computing-Based and Threat Behavior-Aware Smart Prioritization Framework for Cybersecurity Intrusion Detection and Prevention of IEDs in Smart Grids With Integration of Modified LGBM and One Class-SVM Models. IEEE Access, 12.

[20] Cantone, M., Marrocco, C., & Bria, A. (2024). Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study. IEEE Access, 12, 144489–144508. https://doi.org/10.1109/ACCESS.2024.3472907

[21] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116.

[22] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symp. Security and Privacy, 2010, pp. 305–316, doi: 10.1109/SP.2010.25.

[23] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," IEEE Commun. Surveys Tuts., vol. 18, no. 2, pp. 1153–1176, 2016, doi: 10.1109/COMST.2015.2494502.

[24] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," Comput. Security, vol. 86, pp. 147–167, 2019, doi: 10.1016/j.cose.2019.06.005.

[25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[26] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[27] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems, vol. 30, 2017.